전문화된 다중 SLM 협업 시스템에서 최적 통신 프로토콜과 집단 실수 메모리를 활용한 자기 개선 프레임워크 연구

A Self-Improving Framework via Optimal Communication Protocols and Collective Error Memory

2026 년 4 월

민덕기 교수님 + NLP/ 에이전트 랩 발표

목차

1 배경 — 멀티에이전트 SLM 협업의 두 가지 한계

2 라이브 데모 — CompaniAI v1.1

3 시스템 아키텍처 — 4 계층 구조

4 연구 질문 — 3 개의 RQ

5 실험 설계 — 3 개 실험 매트릭스

6 일정 · 학술적 위치 · 마무리

7 Q & A

출발점 ① OLaLA — 가짜뉴스 판별 다중 에이전트

Team A
Stage 1–5
증거 수집

→

Team B
Stage 6–8
이중 대립

→

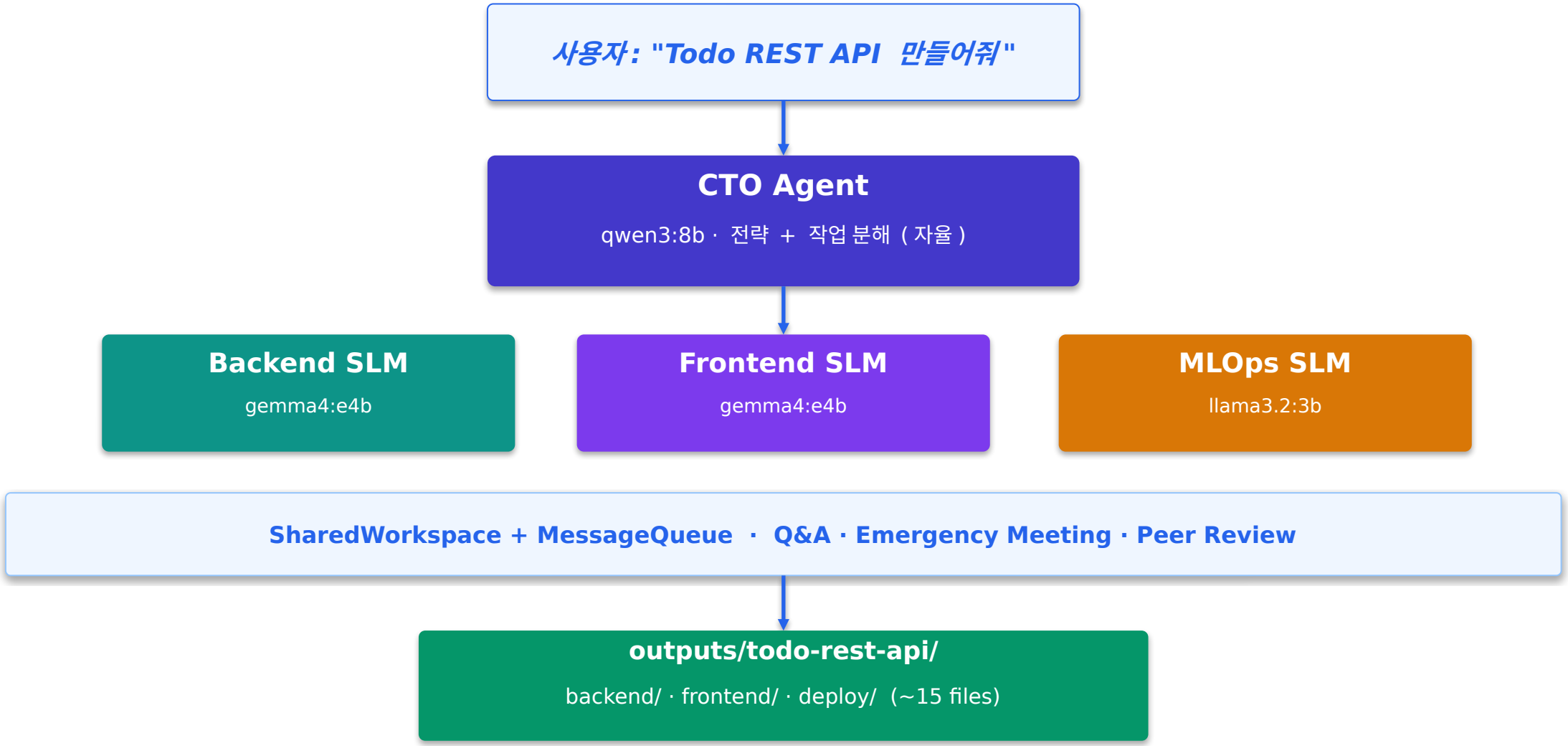
Judge
Stage 9
최종 판정

SLM 두 개가 같은 증거를 ① 지지 / ② 반박 관점 독립 분석

핵심 결과

- 단일 LLM 대비 hallucination ↓ 약 20%
- 정확도 89% 달성
- **역할 분리만으로 SLM 도 충분히 동작 → 첫 의문 도출**

출발점 ② CompaniAI — 사용자 한 문장 → 자동 소프트웨어 생성



GitHub AIBe-lang/compani_ai · 470+ 단위 테스트 · Mac Mini M4 16GB 동작 검증

두 가지 한계 — Research Motivation

한계 ① 단일 LLM 의존 구조

ChatDev / MetaGPT / AutoGPT 등
기존 멀티에이전트 연구는 형식상 여러 에이전트를 두지만, 실제로는 GPT-4 같은 단일 대형 모델을 system prompt 만 바꿔 반복 호출하는 구조.

→ 비용 ↑ · 재현성 ↓ · 단일 모델 한계 그대로

한계 ② 정적인 에이전트 — 경험 누적 부재

실행 중 hallucination / schema violation 이 발생해도 같은 패턴이 다음 실행에서 반복.
ExpeL [Zhao+2024] 은 단일 에이전트 경험 누적은 다뤘으나, 팀 단위 집단 메모리는 거의 미연구 영역.

→ 동일 hallucination 이 task 마다 반복 관측

이 두 한계의 교차점이 본 연구의 출발점입니다.

라이브 데모 — CompaniAI v1.1

🎬 **LIVE DEMO**

Mock 모드 38 초 압축 · 실제는 5–10 분

1

"Todo REST API" 한 문장 입력

2

CTO 전략 → 작업 분해 → SLM 병렬 실행

3

Q&A · 회의 · 피어 리뷰 시각화

4

자동 다운로드 + 산출물 미리보기

[화면 전환 → 데모 페이지]

데모에서 보신 것 + 자연스럽게 도출되는 RQ

✓ **자율성**

한 문장 → 5 개 task 자동 분해

✓ **협업의 깊이**

Q&A · 회의 · 피어 리뷰 (각각 다른 의미)

✓ **추적성**

대화 내용이 실제 코드 결정에 반영
PATCH=Merge Patch · POST=201

이 시스템 운영에서 자연스럽게 떠오른 세 의문 →

RQ1

동일 자원에서
SLM 팀 vs 단일 LLM 우위 ?

RQ2

통신 프로토콜이
정말 최적인가 ?

RQ3

반복 실수를 어떻게
팀 단위로 기억할까 ?

시스템 아키텍처 — 4 계층 구조

L1

오케스트레이션

CTO Agent (qwen3:8b) — 전략 → 분해 → 모니터링 → 메모리 조회

✅ 작동

L2

실행

Backend (gemma4:e4b) · Frontend (gemma4:e4b) · MLOps (llama3.2:3b)
+ asyncio 병렬 + per-role concurrency limiter

✅ 작동

L3

통신 ★ RQ2

SharedWorkspace · MessageQueue · Emergency Meeting · Peer Review
[Protocol A/B/C/D 비교 인터페이스 — 설계 중]

🛠️ 설계 중

L4

집단 실수 메모리 ★ RQ3

Qdrant (벡터) + SQLite (메타) + 자동 규칙 승격 (N≥3)
[석사 1 학기 1 순위 — 미구현]

❌ 미구현

RQ1 동일 FLOPs 에서 SLM 팀이 단일 LLM 대비 우위인 task 유형은 ?

가설

Task 가 (a) 명확히 분해 가능하고 (b) 역할 전문성이 중요할수록 → SLM 팀 우위

Baseline (단일 LLM)

Llama 3.1 70B

단일 호출

총 ~70B 파라미터

모든 컨텍스트 한 번에

Treatment (SLM 팀)

qwen3:8b + gemma4:e4b × 2 + llama3.2:3b

역할 특화 병렬

총 ~17B 파라미터

동일 FLOPs 제어

측정 지표

- 정확도 / 완성도
- End-to-end Latency
- FLOPs / 토큰
- Cross-task 일반화

RQ2 ★ 에이전트 간 통신 프로토콜의 설계 원칙

프로토콜	메시지 인코딩	라우팅	예상 토큰 /task	비고
Protocol A	자유 자연어	broadcast	~2,000	자연스러우나 토큰 비효율
Protocol B	JSON 구조화	targeted	~800	현재 compani_ai 사용
Protocol C	CoT + summary	broadcast	~1,400	투명, 토큰 증가
Protocol D ★	**Importance-filtered (top-K)**	**scored routing**	**~600**	**신규 제안 — 효과 검증 필요**

Protocol D 의 importance score (초기값)

$$\text{importance}(msg) = 0.4 \cdot \text{relevance} + 0.3 \cdot \text{recency} + 0.3 \cdot \text{severity}$$

relevance: cosine sim with task embedding · recency: exp decay · severity: CRITICAL/MAJOR/MINOR/INFO

측정 + 추가 ablation

통신 토큰 · 오류 전파율 · Task 완성률 · Latency + Protocol D 가중치 그리드 서치 ($3^3 \times 3 = 27$ 조합)

RQ3 ★ 집단 실수 메모리 (CEM) 의 효과

ErrorEvent 스키마

error_code
E-LLM-TIMEOUT, E-PARSE-JSON, ...
agent_id · task_id
발생 주체와 컨텍스트
context (embedded)
발생 시점 컨텍스트 임베딩
severity
MINOR / MAJOR / CRITICAL
recurrence_count
같은 패턴 누적 횟수

실수 발생 (자동 캡처)

↓

Qdrant (벡터) + SQLite (메타데이터)

↓

새 task 시작 시 — 유사도 검색 (top-5)
과거 실수 → CTO 프롬프트 주입

↓

🔔 N≥3 회 반복 → 자동 방지 규칙
모든 SLM 의 system prompt 에 고정 삽입

비교 조건 (반복 오류율 감소 곡선 측정)

(a) none	(b) per-agent	(c) shared CEM	(d) (c) + auto-rule

세 RQ 의 위계 — Incremental Design

실험 1 (RQ1) — **단일 LLM vs 다중 SLM**

어느 task 에서 SLM 팀 우위인가?

↓ *(SLM 팀 우위 확인 시)*

실험 2 (RQ2) — **Protocol A / B / C / D 비교**

어떤 통신 방식이 정확성 + 속도?

↓ *(최적 프로토콜 확정 후)*

실험 3 (RQ3) — **CEM 효과 검증**

with 최적 프로토콜 — 반복 오류 감소?

각 실험이 다음 실험의 입력 변수가 되는 단계적 설계

실험 1 단일 LLM vs 다중 SLM (3–4 개월차)

조건	Baseline	Treatment
모델	**Llama 3.1 70B (단일)**	**qwen3:8b + gemma4:e4b × 2 + llama3.2:3b** (SLM 팀, 합산 ~17B)
FLOPs	동일 조건 (제어)	동일 조건 (제어)
Task suite	compani_ai 5 개 reference + HumanEval + OLaLA 일부	compani_ai 5 개 reference + HumanEval + OLaLA 일부
측정	정확도 / 완성도 · End-to-end Latency · FLOPs · 토큰	정확도 / 완성도 · End-to-end Latency · FLOPs · 토큰

예상 결과 시나리오

단순 단일 -step task

→ 단일 LLM 우위

(예: 단일 함수 생성)

복합 다중 모듈 task

→ SLM 팀 우위 (가설)

(예: backend + frontend + deploy)

실험 2 통신 프로토콜 비교 (5–7 개월차)

지표 \ 프로토콜	A	B	C	D ★
통신 토큰 / task	~2,000	~800	~1,400	~600 ?
Task 완성률	?	?	?	?
End-to-end Latency	?	?	?	?
오류 전파율	?	?	?	?

추가 분석

Protocol D 가중치 ablation

$3^3 \times 3 = 27$ 조합 그리드 서치

(관련성 · 최신성 · 심각도)

검증 도메인

compani_ai 5 개 + OLaLA

동일 task suite + 동일 SLM 팀

실험 3 집단 실수 메모리 효과 (8–10 개월차)

메모리 조건	반복 오류율 (10 회 실행 후)	예상 변화
(a) 메모리 없음	*기준선 (baseline)*	—
(b) per-agent 메모리	감소	개인 학습 효과
(c) shared CEM	(b) 보다 큰 감소 (가설)	cross-agent 학습
(d) shared + 자동 규칙	(c) 위 추가 감소 (가설)	선제적 방지

검증 가설

H3-1 (a) → (c) 로 갈수록 N 회 후 오류율 단조 감소
H3-2 (c) vs (d) 비교에서 (d) 가 추가 감소
H3-3 Cross-agent 오류는 (b) 보다 (c) 에서 큰 감소

⚠ **False positive 모니터링 — 잘못 학습된 방지 규칙 비율 (안전장치 검증)**

학술적 위치 — 본 연구의 차별 contribution

기존 연구 대부분은 한 axis 만 다룬다

ChatDev / MetaGPT
단일 LLM, 통신만

ExpeL
단일 에이전트, 메모리만

ProtocolBench (2025)
통신만, 메모리 무시

본 연구는 세 axis 를 동시에 변수로 다룬다

모델
단일 LLM ↔ 다중 SLM

통신
A / B / C / D

메모리
none / per-agent / shared CEM / + rule

특히 (통신) × (메모리) 의 상호작용은 미연구 영역

"통신 품질이 메모리 품질을 결정한다" 는 가설 자체가 새로운 학술 contribution

핵심 참조: ChatDev (ACL 2024) · MetaGPT (ICLR 2024) · ExpeL (AAAI 2024) · ProtocolBench (arXiv 2510.17149) · MemRL (arXiv 2601.03192)

연구 일정 (12 개월)

0 1 2 3 4 5 6 7 8 9 10 11 12

0–2 개월

준비

핵심 논문 정독 + 환경 정비 + compani_ai 코드 정리

3–4 개월

실험 1 (RQ1)

단일 LLM vs SLM 팀 비교 — 워크샵 논문 초안 시도

5–7 개월

실험 2 (RQ2)

Protocol A/B/C/D 비교 + Protocol D 신규 구현 + 가중치 최적화

8–10 개월

실험 3 (RQ3)

CEM 시스템 구현 + 반복 오류 감소 검증

11–12 개월

통합

결과 종합 + 논문 작성 — KCC / ACL workshop / EMNLP findings

연구 한계 + 보완 계획

① 실험 자원 한계

문제

16GB Mac 단독으로 GPT-4 baseline 비교 어려움

보완 계획

→ 연구실 자원 / 클라우드 학생 크레딧 활용

② 검증 도메인의 generalizability

문제

compani_ai (코드) + OLaLA (가짜뉴스) 두 도메인

보완 계획

→ HumanEval / ProtocolBench task suite 추가

③ CEM 의 noise 위험

문제

false positive 방지 규칙이 품질 저하 유발 가능

보완 계획

→ confidence threshold + 외부 검증 (테스트 통과율)

④ Protocol D 가중치의 임의성

문제

초기값 0.4/0.3/0.3 은 직관 기반

보완 계획

→ 그리드 서치 (27 조합) + 메타러닝 최적화

정리

① Baseline 시스템 작동 검증 완료

compani_ai_dev3 — 470+ 단위 테스트 , 16GB Mac 동작
실험 baseline 부재로 인한 시간 낭비 없음

② RQ 의 학술적 위치가 명확

통신 × 메모리 의 상호작용 — 미연구 axis
Protocol D 신규 제안 + 집단 메모리 자동 규칙화

③ 12 개월 일정의 incremental 설계

RQ1 (기반 검증) → RQ2 (통신 최적화) → RQ3 (메모리)
각 실험이 다음 실험의 입력 변수가 되는 구조

④ 검증 도메인 다양성

compani_ai (코드 생성) + OLaLA (가짜뉴스)
+ 표준 벤치 (HumanEval / ProtocolBench)

감사합니다 . · Q & A

[백업] 시스템 상세 — 통신 의미론

SharedWorkspace

- 모든 에이전트는 직접 호출하지 않음
- 오직 SharedWorkspace 만 read/write
- WorkItem 상태 변화 → EventBus broadcast
- 의존성 에이전트는 WorkItem.completed 이벤트까지 polling

MessageQueue Q&A

- point-to-point 메시지
- SQLite-backed, 응답 받기까지 dequeue 보장
- asyncio.wait_for 로 timeout 보호
- 응답이 오지 않으면 escalate or fallback

Emergency Meeting Trigger

- blocking.detected (의존성 충돌)
- peer_review.severity ≥ MAJOR
- stage_gate.failure
- 최종 결정 = CTO 종합 판단

[백업] 데모 비상 시 — 사전 캡처 화면 안내

데모가 멈췄을 때 — 보여줄 화면 4 가지

" 지금 시스템은 사전 정의된 시나리오로 시연하지만, 이게 평소 시스템의 실제 동작 결과 화면입니다."

①	②	③	④
Stage Card 5 개 모두 ✅ 완료	Task 진행 상세 5 개 task 모두 done	결과 카드 + 산출물 트리 exit_code = 0	outputs/todo-rest-api-mock/ backend/main.py 코드

발표 직전 점검 요약

🟢 MOCK 토글 · Stage 카드 5 개 ○ pending · 메시지 로그 비움 · 다운로드 폴더 비움

Teams/Notion/Discord 종료 · 화면 sleep OFF · 외부 모니터 폰트 가독성 확인